SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Preksha Prakash	**Reg. No.:**	1922260048
Date:	22-10-2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

1. Study and Scenario Analysis

1.1 Business Domain & Context

In contemporary enterprise software engineering, distributed applications are expected to remain operational continuously without scheduled maintenance windows. High-availability web services rely on relational and non-relational database management systems that store critical organizational data. As application features evolve, the underlying database schemas must undergo structural changes, such as adding new attributes, splitting existing tables, or modifying data types.

Executing structural alterations directly on a live database often leads to table locking, where incoming read and write transactions are blocked, causing application lag or service outages. Furthermore, when updated database schemas are deployed before all running application instances are refreshed, legacy software components attempt to access non-existent or modified columns, causing runtime crashes. Therefore, the domain of this project belongs to Database DevOps and Site Reliability Engineering (SRE), focusing on automated orchestration techniques that enable continuous database evolution alongside zero-downtime software deployments.

1.2 Identified Stakeholders

The development and operational success of this platform involves several distinct human and system stakeholders. Database Administrators (DBAs) represent a primary administrative stakeholder group tasked with maintaining database stability, verifying lock safety, reviewing automated execution plans, and monitoring performance metrics. DevOps and Release Engineers act as operational stakeholders responsible for integrating database migration hooks into continuous integration and continuous deployment pipelines, managing deployment environments, and ensuring smooth rollbacks.

Backend Software Developers represent the engineering stakeholders who author feature updates alongside schema migration files, requiring rapid feedback on whether their structural changes violate backward compatibility. Finally, end-users of the web application represent passive stakeholders who require uninterrupted access to online services without experiencing downtime, degraded performance, or transaction failures during database updates.

1.3 Existing Challenges

Traditional database migration techniques present severe operational risks in modern high-concurrency environments. Executing data definition language statements on large, highly active tables frequently locks tables for prolonged durations, rejecting incoming traffic and leading to service level agreement violations. Another significant challenge is the version mismatch that arises during rolling application updates. When application servers update incrementally across server farms, older instances running previous codebase versions continue querying the database

simultaneously with newer instances. If a column is dropped or renamed prematurely, legacy instances crash instantly.

Additionally, manual rollback procedures during unexpected deployment failures are extremely dangerous, as attempting to revert a schema change manually can result in orphan records or permanent data loss. The absence of automated static analysis tools in standard integration workflows further aggravates these issues, leaving development teams unaware of breaking schema changes until they hit production databases.

1.4 Primary System Objectives

The proposed platform addresses these challenges by establishing an automated framework for safe, non-blocking schema evolution. The primary objective is to automate static schema analysis and pre-deployment testing, ensuring that every database change is verified for backward compatibility before execution.

The system implements multi-version deployment patterns, such as the expand-and-contract model, allowing legacy and updated application instances to operate concurrently on the database without errors. Furthermore, the system provides continuous health monitoring and automated emergency recovery mechanisms that safely halt and revert migrations if database performance metrics breach safe operating limits. By integrating directly into continuous integration tools, the platform minimizes manual operational effort while providing complete auditability across all schema evolution stages.

2. Requirements Identification

2.1 Functional Requirements

The platform must incorporate a comprehensive set of functional capabilities to govern the database schema lifecycle. The system must parse SQL migration scripts and apply static analysis rules to identify potentially destructive schema operations, such as dropping columns, renaming tables, or adding non-nullable fields without default values. It must execute automated backward compatibility tests by simulating how both previous and upcoming application code versions interact with the proposed schema modifications. To prevent table locking during deployment, the platform must manage multi-stage execution workflows following the expand-and-contract pattern, wherein structural additions are deployed first, application instances are updated sequentially, and deprecated schema elements are removed only after legacy code is completely decommissioned.

The platform must also offer continuous health tracking by monitoring real-time database indicators, including query latencies, active connection counts, lock wait durations, and replication lag during migration execution. If critical health thresholds are violated or error rates spike during deployment, the platform must automatically trigger non-destructive rollback sequences that restore application stability without corrupting newly created production records. To ensure smooth adoption into existing software delivery pipelines, the system must expose command-line interfaces

and webhook listeners that communicate with external continuous integration servers. Lastly, the system must retain a detailed migration history logging execution timestamps, applied schema differences, performance metrics, and authorizing user identities for security and regulatory auditing.

2.2 Non-Functional Requirements

To operate reliably in enterprise environments, the platform must fulfill rigorous quality and performance standards. High availability is essential, requiring the migration orchestrator to maintain ninety-nine point nine nine percent system availability so that deployment automation remains accessible at all times. System performance must be optimized such that pre-deployment static analysis and compatibility checks for up to fifty migration files complete within fifteen seconds, preventing build pipeline bottlenecks.

Security requirements dictate that all sensitive database credentials, API keys, and connection strings must be encrypted using standard AES-256 encryption at rest and protected via TLS 1.3 encryption during transit. In terms of system reliability, if a network failure or node failure occurs midway through a deployment, the platform must maintain system state using distributed transaction locking to prevent partial, uncoordinated schema updates. The user interface must emphasize clarity by presenting visual dependency graphs, color-coded risk assessments, and real-time execution dashboards. Finally, scalability requirements mandate that the platform must concurrently manage and coordinate migration workflows across up to one hundred independent database clusters without performance degradation.

3. Actors and Use Case Modeling

3.1 Identification of Actors

System actors are categorized into primary human actors who interact directly with the management interfaces and secondary external system actors that interface via automated protocols. Primary human actors include Software Developers, who write feature updates and submit associated migration scripts; Database Administrators, who define safety rules, review automated risk reports, and authorize migration plans; and DevOps Engineers, who configure deployment pipelines and monitor environment health. Secondary system actors include the Continuous Integration Pipeline, which invokes migration analysis and execution workflows automatically, and the Target Database Cluster, which executes approved non-blocking DDL commands and returns real-time operational metrics.

3.2 List of Use Cases

The system functionality is organized into six primary use cases:

1. **UC-01 Submit Migration Script:** Software developers submit new schema change scripts alongside

application source code updates.

2. **UC-02 Perform Pre-Deployment Compatibility Check:** The static analysis engine scans submitted scripts for unsafe DDL patterns and evaluates backward compatibility.

3. **UC-03 Approve Schema Evolution Plan:** Database administrators inspect generated risk scores and approve or reject proposed schema updates.

4. **UC-04 Execute Zero-Downtime Migration:** The platform orchestrates phased expand-and-contract updates on the target database cluster without interrupting application availability.

5. **UC-05 Monitor Real-Time Database Health:** The system continuously collects health indicators from the database during migration execution.

6. **UC-06 Trigger Automated Rollback:** The platform halts migration and restores a safe state if performance limits are exceeded during deployment.

3.3 Case Diagram

Actors	Associated Use Cases	Description / Interaction
Developer	UC-01: Submit Migration Script	Submits new schema changes alongside app code.
CI/CD Pipeline	UC-02: Perform Compatibility Check	Automatically tests migration files for breaking changes.
DBA	UC-03: Approve Schema Plan	Reviews safety reports and authorizes execution.
Target Database Cluster	UC-04: Execute Zero-Downtime Migration	Processes expand/contract scripts and streams health metrics.
DevOps Engineer	UC-06: Trigger Automated Rollback	Halts deployment and reverts state if error thresholds fail.

4. Detailed SRS Specification

4.1 Scope of the System

The platform bridges the operational gap between rapid application deployment and database stability. The software provides automated governance across the entire database evolution lifecycle, starting from initial static analysis in developer environments, continuing through approval and deployment orchestration, and concluding with continuous

health monitoring and auditing. By enforcing non-blocking structural modification strategies, the platform eliminates planned downtime windows and reduces human error during production deployments.

4.2 Comprehensive Use Case Narrative Descriptions

Detailed Narrative for UC-02: Perform Pre-Deployment Compatibility Check

The compatibility check process initiates when a developer commits a new migration file to a controlled source repository branch. The external continuous integration pipeline intercepts the repository event and dispatches an automated request containing the raw migration SQL file to the platform's analysis engine.

Upon receiving the payload, the platform parses the SQL code into an abstract syntax tree and evaluates it against predefined safety rules. The engine scans for breaking operations, such as immediate table drops, column renames, or non-default column additions, while simultaneously querying the active schema catalog to verify whether the change breaks legacy application queries.

Once analysis completes, the system generates a structured risk assessment report containing a numerical safety score, identified policy warnings, and recommended corrective actions. If no critical violations are detected, the system sends a success signal back to the integration pipeline, allowing the build process to proceed to the review stage.

Detailed Narrative for UC-04: Execute Zero-Downtime Migration

The execution workflow begins after a database administrator reviews and approves an analyzed schema modification plan. The platform initiates the expand phase by applying additive, non-blocking structural changes to the live database, such as creating new columns or shadow tables, while leaving existing structures untouched.

During this phase, trigger mechanisms or dual-writing strategies ensure that new data is recorded across both old and new schema components simultaneously. The platform verifies that data synchronization is functioning correctly before notifying the deployment orchestrator to begin rolling updates for the application server instances.

As legacy application instances are decommissioned and new instances assume production traffic, the platform verifies that zero active connections are referencing the legacy schema elements. Once complete application transition is confirmed, the system enters the contract phase, safely removing deprecated tables or columns and completing the schema evolution cycle without incurring any user-facing downtime.

4.3 Data and External Interface Requirements

The system requires structured data representations to manage migration workflows and operational logs. Core data structures include the Migration Script entity, which tracks script identifiers, version numbers, author details,

cryptographic checksums, execution status, and submission timestamps. The Execution Log entity records details regarding individual migration runs, including environment names, execution start and end times, lock wait durations, impacted row counts, and final status flags. The Safety Policy entity stores rule definitions, rule severity classifications, and threshold parameters used by the static analysis parser.

External software interfaces require native database connectivity drivers for engines like PostgreSQL and MySQL, standard webhook and RESTful HTTP endpoints for continuous integration tools such as GitHub Actions and Jenkins, and metric exporter interfaces for system monitoring tools like Prometheus and Grafana.

5. Assumptions, Constraints, and Requirement Validation

5.1 Project Assumptions

The architecture relies on several operational assumptions regarding the deployment environment. It is assumed that the target relational database management systems support transactional DDL or online schema modification strategies, such as shadow tables or online index builds. It is further assumed that backend application developers adhere to dual-writing principles when designing complex, multi-stage schema transformations. Finally, the platform assumes stable, low-latency network communication, with round-trip latency between the orchestrator and target database clusters remaining below ten milliseconds under normal operating conditions.

5.2 System Constraints

System implementation is subject to technical, operational, economic, and time constraints. Technically, the platform must operate using standard database credentials without requiring elevated superuser privileges on production instances. Operationally, background data backfill processes on large tables exceeding one hundred million records must dynamically throttle execution based on host CPU and IO performance metrics to prevent production performance degradation.

Economically, the core static analysis and SQL parsing modules must be built using open-source parsing libraries to minimize software licensing costs. From a time perspective, the specification, documentation, and prototype verification must be finalized within the assigned academic evaluation timeframe.

5.3 Requirement Validation and Consistency Analysis

To ensure complete specification quality, all requirement statements were validated against stakeholder goals and industry standard Software Requirement Specification guidelines. Requirement completeness was verified by ensuring every phase of the schema evolution lifecycle—from authoring to rollback—is addressed by specific functional requirements. Ambiguity was minimized by eliminating subjective terms and replacing them with

quantifiable operational metrics, such as explicit execution time limits and availability percentages.

Consistency checks were performed to align non-functional safety goals with the automated recovery triggers specified in the use case models, ensuring that performance degradation detected during real-time monitoring directly invokes rollback protocols without system contradiction.

6. Conclusion and Future Enhancements

6.1 Conclusion

The design and specification of the **Zero-Downtime Database Migration and Schema Evolution Platform** address one of the most critical operational bottlenecks in modern, high-availability software systems. Traditional database deployment strategies frequently introduce severe operational risks, such as database table locks, service degradation, schema version mismatches, and system crashes during active user sessions. By applying structured Software Engineering principles and scenario-based requirement analysis, this Software Requirement Specification (SRS) establishes a robust framework that reconciles continuous application delivery with database stability.

Through the implementation of pre-deployment static analysis, multi-stage expand-and-contract execution workflows, real-time database health monitoring, and automated safety rollbacks, the platform eliminates planned maintenance windows and prevents data corruption. The integration of well-defined functional and non-functional requirements ensures that all primary and secondary stakeholders—including Developers, Database Administrators, DevOps Engineers, and automated CI/CD pipelines—interact seamlessly with the database infrastructure under strict security and performance constraints. Ultimately, the system guarantees 99.99% operational uptime while providing an audited, traceable history of all schema evolution activities.

6.2 Future Enhancements

While the current platform specification comprehensively satisfies enterprise zero-downtime requirements, future iterations can introduce advanced capabilities to further optimize database management:

- **AI-Driven Predictive Risk Scoring:** Incorporating machine learning models to analyze historical query execution plans and database traffic trends, enabling predictive detection of lock risks before executing DDL commands on large tables.
- **Cross-Cloud Multi-Region Schema Synchronization:** Expanding orchestrator capabilities to seamlessly coordinate asynchronous schema evolution across distributed multi-region and multi-cloud database deployments.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5

Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25